

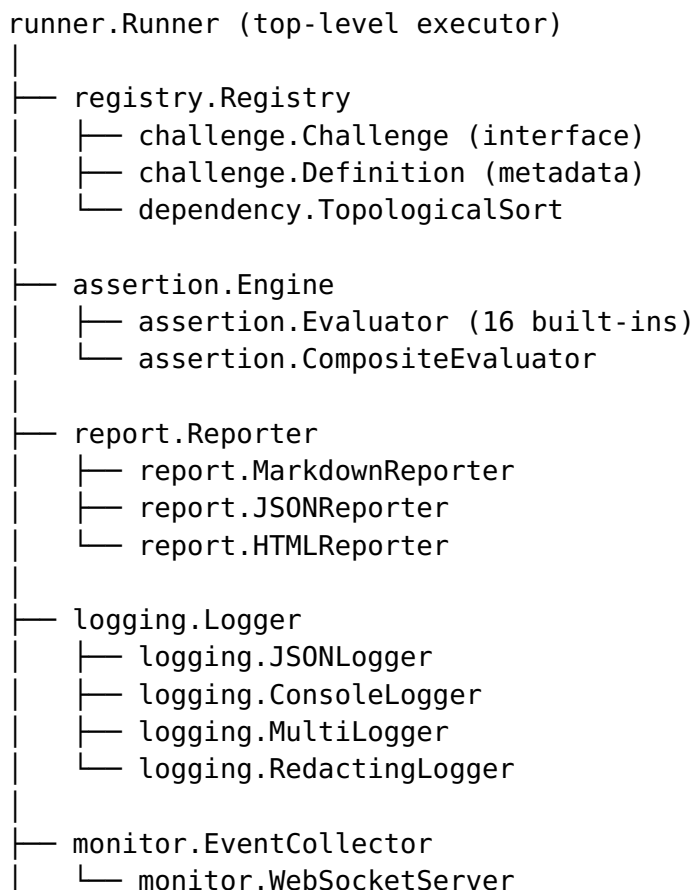
ARCHITECTURE

Challenges Module - Architecture

Design Philosophy

The Challenges module provides a **generic, extensible framework** for defining, executing, and reporting on structured test scenarios. It is designed to be used as a library by any Go application that needs to run and validate complex test suites.

Package Dependency Graph



```
|
└─ plugin.PluginRegistry

challenge.Challenge (interface)
└─ challenge.BaseChallenge (template method)
└─ challenge.ShellChallenge (bash adapter)
└─ [application-specific challenges]

infra.InfraProvider
└─ infra.ContainersAdapter
    └─ digital.vasic.containers/pkg/lifecycle
```

Design Patterns

Template Method

BaseChallenge provides the lifecycle skeleton (Configure → Validate → Execute → Cleanup). Concrete challenges embed BaseChallenge and override Execute().

Strategy

- report.Reporter with Markdown/JSON/HTML implementations
- assertion.Evaluator functions as interchangeable strategies

Registry

- registry.Registry for challenges and definitions
- plugin.PluginRegistry for plugins
- assertion.Engine as an evaluator registry

Adapter

- challenge.ShellChallenge adapts bash scripts to the Challenge interface
- infra.ContainersAdapter bridges to the Containers module

Decorator

- logging.RedactingLogger wraps any Logger with secret redaction

Observer

- monitor.EventCollector captures challenge events for live monitoring

Functional Options

- `runner.NewRunner(WithRegistry(), WithLogger(), WithTimeout())`

Challenge Execution Lifecycle

1. **Load:** Get challenge from registry
2. **Setup:** Create results directory structure
3. **Configure:** Pass runtime config to challenge
4. **Validate:** Check dependencies and prerequisites
5. **Execute:** Run with timeout (context deadline)
6. **Evaluate:** Run assertions against results
7. **Report:** Generate reports (MD/JSON/HTML)
8. **Cleanup:** Release resources

Dependency Ordering

Uses Kahn's algorithm for topological sorting:

- Build in-degree map from dependency graph
- Start with zero-in-degree challenges
- Process and decrement dependents
- Detect cycles (ordered count $\neq$ total count)

Thread Safety

- `registry.DefaultRegistry` uses `sync.RWMutex`
- `assertion.DefaultEngine` uses `sync.RWMutex`
- `runner.ParallelRunner` uses goroutines + semaphore for concurrency control
- `monitor.EventCollector` uses channels for event delivery